Python并发编程

松松 20200403

目录一览

Python并发编程
- 1 程序提速方法
  - 1.1 多线程并发
  - 1.2 多CPU并行
  - 1.3 多机器并行
- 2 python并发编程三种方式
  - 2.1 什么是CPU密集型计算、IO密集型计算？
    - 2.1.1 CPU密集型（CPU-bound）
    - 2.1.2 IO密集型 （I/O bound）
  - 2.2 多线程、多进程、多协程的比对
  - 2.3 全局解释器锁GIL
- 3 python多线程
  - 3.1 创建多线程的方法
  - 3.2 线程安全问题
  - 3.3 线程池
- 4 python多进程
  - 4.1 多进程概述
  - 4.2 进程之间共享数据（全局变量）
    - 4.1.1 内存共享
      - 4.1.1.1 共享数值示例
      - 4.1.1.2 共享数组示例
    - 4.1.2 服务器进程
      - 4.1.2.1 共享元组列表示例
      - 4.1.2.2 共享Pandas的Dataframe示例
  - 4.3 数据传递
    - 4.3.1 Queue
    - 4.3.2 pipe
  - 4.4 进程同步（加锁）
  - 4.5 进程池化
  - 4.6 多进程运行含有任意个参数的函数

1 程序提速方法

传统代码是单线程串行，但是python有以下三种提速方法，值得注意的是并发执行只是高效利用CPU而已，并行效果甚微。

1.1 多线程并发

对应技术：threading（多线程），利用CPU和IO可以同时执行的原理，避免CPU干巴巴等待IO完成；asyncio（异步IO），在单线程利用CPU和IO同时执行的原理，实现函数异步执行。

1.2 多CPU并行

对应技术：multiprocessing（多进程），利用多核心CPU功能，真正执行并行任务。

1.3 多机器并行

对应技术: hadoop/hive/spark

attention：常用多进程和多线程用于加速。

2 python并发编程三种方式

2.1 什么是CPU密集型计算、IO密集型计算?

2.1.1 CPU密集型（CPU-bound）

CPU密集型也叫计算密集型，即CPU占用率极高！例如压缩解压缩、加密解密。

2.1.2 IO密集型 （I/O bound）

IO密集型就是系统大部分工作用于数据读写，CPU占用率很低。例如：文件处理程序、网络爬虫程序、数据库读写等。

2.2 多线程、多进程、多协程的比对

名称	python技术	优点	缺点	适用场景
多进程（Process）	multiprocessing	可以利用多核CPU计算	占用资源最多，可启动数目比线程少。	CPU密集型计算
多线程（Thread）	threading	**相比进程：**更轻量、占用资源少。	**相比进程：**多线程只能并发执行，不能利用多CPU（GIL：全局解释器锁）；**相比协程：**启动数目有限制，占用内存资源，有线程切换开销。	IO密集型计算；同时运行的数目不多。
多协程（Coroutine）	asynocio	一个线程中可以启动N个协程；内存开销最小，启动协程数量最多。	支持库有限，代码实现复杂。	IO密集型计算，需要超多任务运行。

attention：一个进程可以启动N个线程，一个线程可以启动N个协程。

2.3 全局解释器锁GIL

python慢的原因：1、动态解释性语言，边解释边运行；2、GIL。

GIL全局解释器锁（Global Interpreter Lock）：用于同步线程的一种机制，它使得任何时刻只有一个线程执行任务计算。

GIL作用：解决多线程之间数据完整性和状态同步问题。

attention: CPU在I/O期间会释放GIL，因此可以实现CPU和IO的并行

3 python多线程

3.1 创建多线程的方法

```
# 1:准备一个函数
def myFunc(a,b):
    pass

# 2：创建一个线程
import threading
t = threading.Thread(target=myFunc, args=(100,200)) # 创建进程对象，传入函数名称，参数元组

# 3：启动线程
t.start()

# 4：等待结束
t.join()
```

3.2 线程安全问题

线程安全是指安全处理共享变量之间的相互操作，不在同一时刻对同一变量/内存进行写操作，针对共享内存访问问题，采用Lock解决。

```
# 方法1： try-finally
import threading
lock = threading.Lock()
lock.acquire()
try:
    # do something
finally:
    lock.realse()

# 方法2： with
import threading
lock = threading.Lock()
with lock:
    # do something
```

3.3 线程池

线程池原理：线程重用，减去新建/终止线程的时间开销。

好处：提升性能，适用于突发性大量请求；代码简洁；防御功能，避免创建大量线程；

```
from concurrent.futures import ThreadPoolExecutor, as_completed

# 方法1：map函数，很简单，入参函数是对应的。
with ThreadPoolExecutor() as pool:
    results = pool.map("需要并行的函数名", "传入函数元组的列表！注意是很多个参数元组的列表！！")

    for result in results:
        print(result)

# 方法2：future模式，功能强大，如果使用as_completed则顺序不一定
with ThreadPoolExecutor() as pool:
    futures = [pool.submit("函数名称", index)
               for index in "函数元组的列表"]

    for future in futures:
        print(future.result())

    for future in as_complete(futures): # 好处：不用等待！
        print(future.result())
```

attention：推荐使用进程池

4 python多进程

4.1 多进程概述

多进程是什么?

多进程指的是操作系统同时支持多个处理器的能力。在支持多任务操作系统中，一个应用程序会被分解成多个独立运行的较小的程序。操作系统会将这些线程分配到多核处理器，以提升系统性能。

为什么需要多进程?

假设我们的计算机只有一个单核的处理器，然后同时被分配了几个任务，那么它就不得不在各个任务中来回切换，短暂地执行其中一个任务，然后中断，然后短暂地执行下一个任务，以保持所有的进程都在运行。这就像一个厨师在做面条，切几秒钟菜，跑去搅几下面，再赶紧查看下汤。

所以同时需要完成的任务越多，同时跟踪所有的任务就越困难。这就是多进程的必要性，也是多核处理器的威力所在。

一个支持多进程的操作系统可以做到：

- 同时指挥多个CPU，即拥有一个以上的CPU的计算机
- 指挥多核CPU，即拥有两个及以上的独立处理单元的CPU

4.2 进程之间共享数据（全局变量）

参考自：Python 进程之间共享数据(全局变量)

参考自：Python 多进程之间共享变量

python多进程加速通过定义全局变量glob即可，而进程之间是相互独立的执行单元，因此共享全局变量不可行。python标准库，提供了进程共享内存接口，一种是process多进程，一种是pool进程池方式。使用 Process 定义的多进程之间共享变量可以直接使用 multiprocessing 下的 Value，Array，Queue 等，如果要共享 list，dict，可以使用强大的 Manager 模块。

4.1.1 内存共享

4.1.1.1 共享数值示例

-----》 Value

```
import multiprocessing


def func(num, changedNumber):
    with num.get_lock():
        num.value = changedNumber
    # 子进程改变数值的值，主进程跟着改变


if __name__ == "__main__":
    num = multiprocessing.Value("d", 10.0)
    # d表示数值,主进程与子进程共享这个value。（主进程与子进程都是用的同一个value）
    print(type(num), num.value)

    for changedNumber in [19, 23, 4]:
        p = multiprocessing.Process(target=func, args=(num, changedNumber,))
        p.start()
        p.join()
        print(type(num), num.value)

    print(type(num), num.value)

"""
out:
    <class 'multiprocessing.sharedctypes.Synchronized'> 10.0
    <class 'multiprocessing.sharedctypes.Synchronized'> 19.0
    <class 'multiprocessing.sharedctypes.Synchronized'> 23.0
    <class 'multiprocessing.sharedctypes.Synchronized'> 4.0
    <class 'multiprocessing.sharedctypes.Synchronized'> 4.0
"""
```

4.1.1.2 共享数组示例

-----》 Array

```
import multiprocessing


def func(num, changedNumber):
    num[2] = changedNumber
```

```
    # 子进程改变数组，主进程跟着改变


if __name__ == "__main__":
    num = multiprocessing.Array("i", [1, 2, 3, 4, 5])
    # 主进程与子进程共享这个数组
    print(type(num), num[:])

    changedNumber = 1000
    p = multiprocessing.Process(target=func, args=(num, changedNumber))
    p.start()
    p.join()

    print(type(num), num[:])

"""
out:
    <class 'multiprocessing.sharedctypes.SynchronizedArray'> [1, 2, 3, 4, 5]
    <class 'multiprocessing.sharedctypes.SynchronizedArray'> [1, 2, 1000, 4, 5]
"""
```

4.1.2 服务器进程

每当python程序启动时，同时也会启动一个服务器进程。随后，只要我们需要生成一个新进程，父进程就会连接到服务器并请求它派生一个新进程。这个服务器进程可以保存Python对象，并允许其他进程使用代理来操作它们。

multiprocessing模块提供了能够控制服务器进程的Manager类。所以，Manager类也提供了一种创建可以在不同流程之间共享的数据的方法。

服务器进程管理器比使用共享内存对象更灵活，因为它们可以支持任意对象类型，如列表、字典、队列、值、数组等。此外，单个管理器可以由网络上不同计算机上的进程共享。

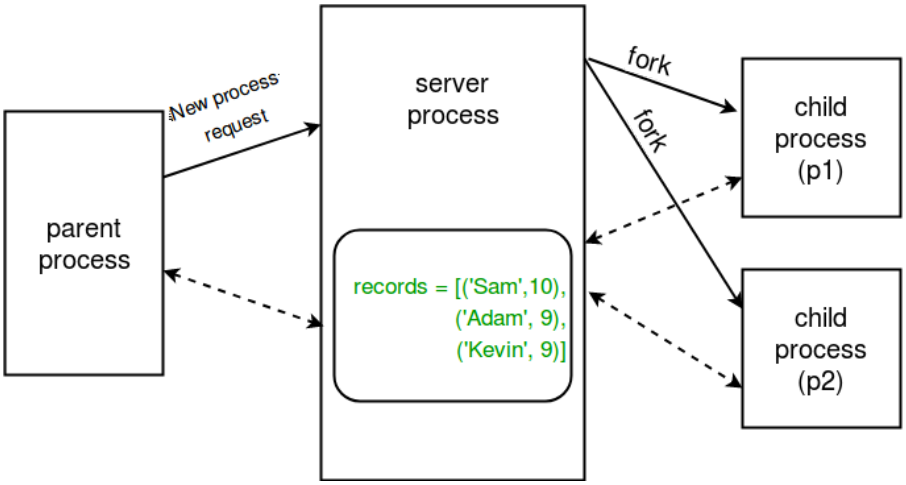
但是，服务器进程管理器的速度比使用共享内存要慢。

一个例子：

```
import multiprocessing

def print_records(records):
    for record in records:
        print("Name: {0}\nScore: {1}\n".format(record[0], record[1]))

def insert_record(record, records):
    records.append(record)
    print("New record added!\n")

if __name__ == '__main__':
    with multiprocessing.Manager() as manager:
        # 首先创建了一个manager对象,在with语句下的所有行，都是在manager对象的范围内的。
        # 接下来我们使用这个manager对象创建了列表
        records = manager.list([('Sam', 10), ('Adam', 9), ('Kevin',9)])
        new_record = ('Jeff', 8)

        # 最后我们创建了进程p1（用于在records列表中插入一条新的record）和p2（将records打印出来），并将records作为参数进行传递。
        p1 = multiprocessing.Process(target=insert_record, args=(new_record, records))
```

```
        p2 = multiprocessing.Process(target=print_records, args=(records,))

        # running process p1 to insert new record
        p1.start()
        p1.join()

        # running process p2 to print records
        p2.start()
        p2.join()
"""
out:
    New record added!

    Name: Sam
    Score: 10

    Name: Adam
    Score: 9

    Name: Kevin
    Score: 9

    Name: Jeff
    Score: 8
"""
```

上例服务器进程的概念用下图总结下：



Text

4.1.2.1 共享元组列表示例

-----》 dict、list

A：普通数据共享方式

```
import multiprocessing


def func(mydict, mylist):
    mydict["index1"] = "aaaaaa"
    # 子进程改变dict,主进程跟着改变
    mydict["index2"] = "bbbbbb"
    mylist.append(11)
    # 子进程改变List,主进程跟着改变
    mylist.append(22)
```

```
    mylist.append(33)


if __name__ == "__main__":
    mydict = multiprocessing.Manager().dict()
    # 主进程与子进程共享这个字典
    mylist = multiprocessing.Manager().list(range(5))
    # 主进程与子进程共享这个List

    p = multiprocessing.Process(target=func, args=(mydict, mylist))
    p.start()
    p.join()

    print(type(mylist), mylist)
    print(type(mydict), mydict)
"""
out:
    <class 'multiprocessing.managers.ListProxy'> [0, 1, 2, 3, 4, 11, 22, 33]
    <class 'multiprocessing.managers.DictProxy'> {'index1': 'aaaaaa', 'index2':
'bbbbbb'}
"""
```

B：采用进程池共享元组和列表

```
from multiprocessing import Pool, Manager


def func(my_list, my_dict):
    my_list.append(10)
    my_list.append(11)
    my_dict['a'] = 1
    my_dict['b'] = 2


if __name__ == '__main__':
    manager = Manager()
    my_list = manager.list()
    my_dict = manager.dict()

    pool = Pool(processes=2)
    for i in range(0, 2):
        pool.apply_async(func, (my_list, my_dict))
    pool.close()
    pool.join()

    print(my_list)
    print(my_dict)
```

还有一点需要注意，在共享 list 时，像下面这样写 func() 是不起作用的。这样写相当于重新定义了一个局部变量，并没有作用到原来的 list 上，必须使用 append，extend 等方法。

```
def func(my_list, my_dict):
    my_list = [10, 11]
    my_dict['a'] = 1
    my_dict['b'] = 2
```

4.1.2.2 共享Pandas的Dataframe示例

```
import pandas as pd
import numpy as np
import multiprocessing

def add_new_derived_column(ns):
    dataframe2 = ns.df
    dataframe2['new_column']=dataframe2['A']+dataframe2['B'] / 2
    ns.df = dataframe2

if __name__ == "__main__":

    ns = multiprocessing.Manager().Namespace() #创建共享命名空间管理器
    dataframe = pd.DataFrame(np.random.randn(100000, 2), columns=['A', 'B'])
    ns.df = dataframe

    print (ns.df.head())

    # then I pass the "shared_df_obj" to Mulitiprocessing.Process object
    process=multiprocessing.Process(target=add_new_derived_column, args=(ns,))
    process.start()
    process.join()

    print (ns.df.head())

"""
out:
                  A         B
    0 -0.888850 -0.218942
    1  0.355486  0.115262
    2 -1.680027  1.209736
    3 -0.555409 -0.432712
    4 -0.453406 -1.102396

              A         B  new_column
    0 -0.888850 -0.218942   -0.998321
    1  0.355486  0.115262    0.413117
    2 -1.680027  1.209736   -1.075159
    3 -0.555409 -0.432712   -0.771765
    4 -0.453406 -1.102396   -1.004604
"""
```

4.3 数据传递

为了能使多个流程能够正常工作，常常需要在它们之间进行一些通信，以便能够划分工作并汇总最后的结果。multiprocessing模块支持进程之间的两种通信通道：Queue和Pipe。

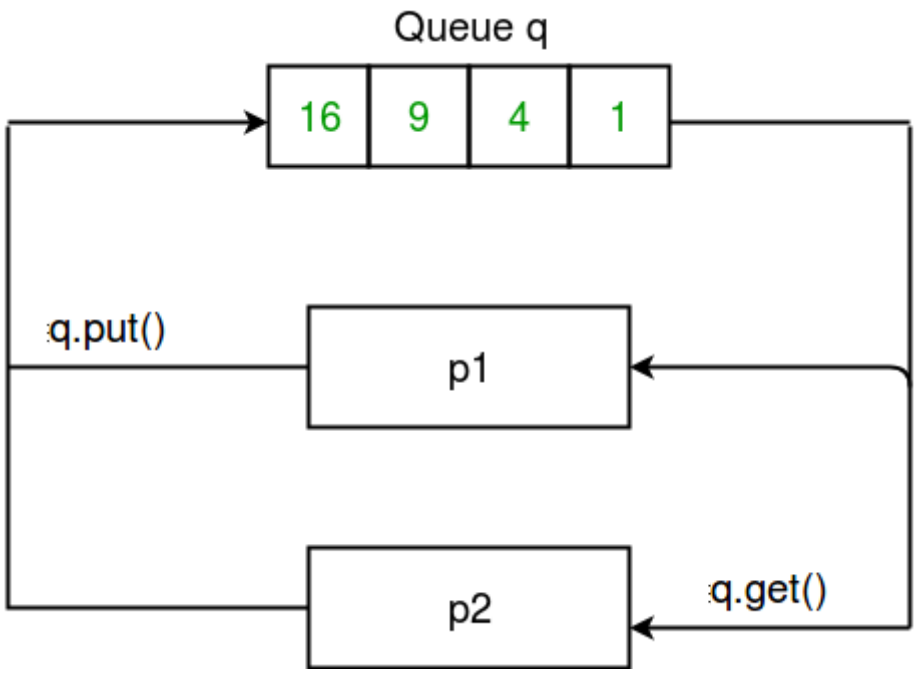
4.3.1 Queue

使用队列来回处理多进程之间的通信是一种比较简单的方法。任何Python对象都可以使用队列进行传递。示例如下：

```
import multiprocessing
```

```python
def square_list(mylist, q):
    # 将队列平方，并把结果放在队列q
    for num in mylist:
        q.put(num * num)

def print_queue(q):
    print("Queue elements:")
    while not q.empty():
        print(q.get())
    print("Queue is now empty!")

if __name__ == "__main__":
    mylist = [1,2,3,4]

    # 1:创建queue对象
    q = multiprocessing.Queue()
    # 2：创建2个进程
    p1 = multiprocessing.Process(target=square_list, args=(mylist, q))
    p2 = multiprocessing.Process(target=print_queue, args=(q,))
    # 3：运行进程1，进程1执行square_list函数，函数对列表元素平方，并把结果压入队列q
    p1.start()
    p1.join()
    # 4：运行进程1，得到queue元素
    p2.start()
    p2.join()
"""
out:
    Queue elements:
    1
    4
    9
    16
    Queue is now empty!
"""
```

示例如图：

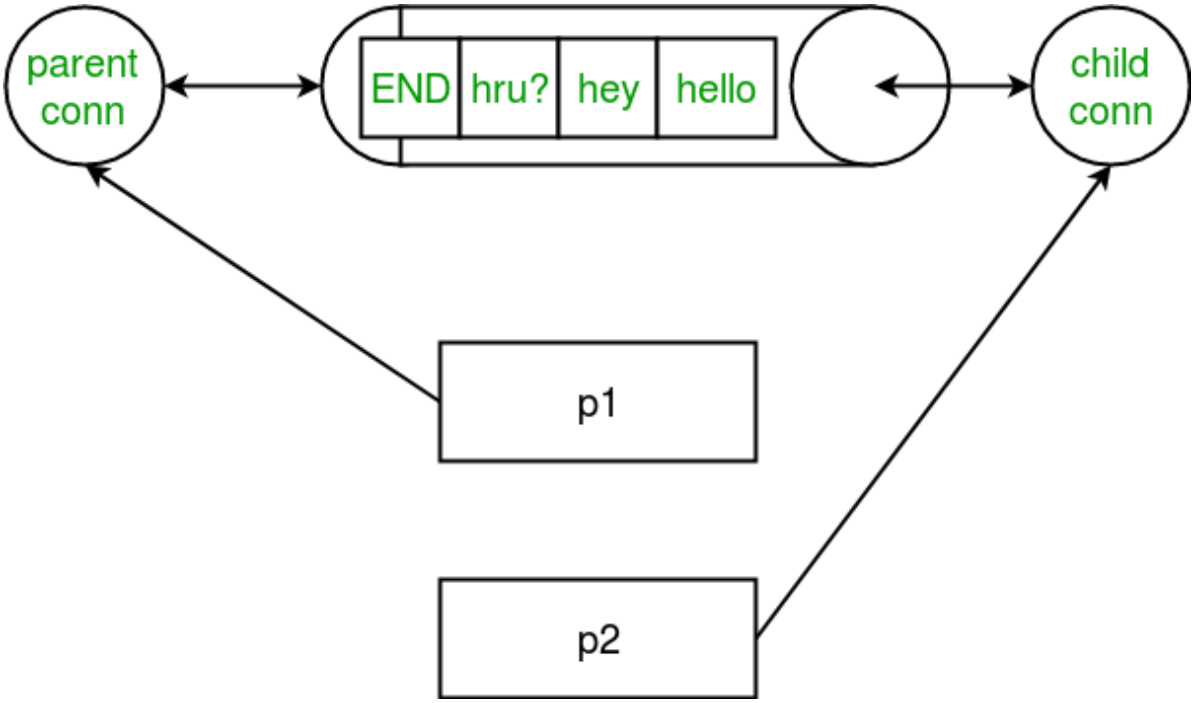


4.3.2 pipe

一个Pipe对象只能有两个端点。因此，当进程只需要双向通信时，它会比Queue对象更好用。multiprocessing模块提供了`Pipe()`函数，该函数返回由管道连接的一对连接对象。`Pipe()`返回的两个连接对象分别表示管道的两端。每个连接对象都有`send()`和`recv()`方法。示例如下：

```
import multiprocessing

def sender(conn, msgs):
    for msg in msgs:
        conn.send(msg)
        print("Sent the message: {}".format(msg))
    conn.close()

def receiver(conn):
    while 1:
        msg = conn.recv()
        if msg == "END":
            break
        print("Received the message: {}".format(msg))

if __name__ == "__main__":
    msgs = ["hello", "hey", "hru?", "END"]

    """创建pipe双端对象。该对象返回了一对管道两端的两个连接对象。然后使用send()方法和recv()方法进行信息的传递。
    要注意的是，如果两个进程(或线程)同时尝试从管道的同一端读取或写入管道中的数据，则管道中的数据可能会损坏。
    不过不同的进程同时使用管道的两端是没有问题的。
    还要注意，Queue对象在进程之间进行了适当的同步，但代价是增加了计算复杂度。因此，Queue对象对于线程和进程是相对安全的。"""
    parent_conn, child_conn = multiprocessing.Pipe()

    # 2：创建两个进程
    p1 = multiprocessing.Process(target=sender, args=(parent_conn,msgs))
    p2 = multiprocessing.Process(target=receiver, args=(child_conn,))

    # 3：启动两个进程
    p1.start()
    p2.start()

    # 4：等待进程结束
    p1.join()
    p2.join()
"""
out:
    Sent the message: hello
    Sent the message: hey
    Sent the message: hru?
    Sent the message: END
    Received the message: hello
    Received the message: hey
    Received the message: hru?
"""
```

图示如下：



4.4 进程同步（加锁）

进程同步是一种确保两个或多个并发进程不同时执行某些特定的程序段(关键段)的一种机制。所谓关键段，是指程序中访问共享数据的部分。如果不同的进程同时访问共享数据的话，可能会引起资源竞争的问题。所谓资源竞争，就是多个进程同时访问共享数据，并且当它们试图同时更改该数据时，就会出现竞争。这种情况下，变量的修改结果就是不可预测的了。我们用一个程序来展示资源竞争问题：

```
import multiprocessing

# 取钱
def withdraw(balance):
    for _ in range(10000):
        balance.value = balance.value - 1

# 存钱
def deposit(balance):
    for _ in range(10000):
        balance.value = balance.value + 1

def perform_transactions():
    balance = multiprocessing.Value('i', 100)
    p1 = multiprocessing.Process(target=withdraw, args=(balance,))
    p2 = multiprocessing.Process(target=deposit, args=(balance,))
    p1.start()
    p2.start()
    p1.join()
    p2.join()

    print("Final balance = {}".format(balance.value))

if __name__ == "__main__":
    for _ in range(10):
        perform_transactions()

"""out:
    Final balance = 1852
    Final balance = -2323
```

```
    Final balance = -126
    Final balance = -290
    Final balance = 1038
    Final balance = -3905
    Final balance = -947
    Final balance = 1312
    Final balance = 786
    Final balance = 2472
"""
```

共享资源出错，资源加锁！

```
import multiprocessing

def withdraw(balance, lock):
    for _ in range(10000):
        lock.acquire()
        balance.value = balance.value - 1
        lock.release()

def deposit(balance, lock):
    for _ in range(10000):
        lock.acquire()
        balance.value = balance.value + 1
        lock.release()

def perform_transactions():
    balance = multiprocessing.Value('i', 100)
    lock = multiprocessing.Lock()
    p1 = multiprocessing.Process(target=withdraw, args=(balance,lock))
    p2 = multiprocessing.Process(target=deposit, args=(balance,lock))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
    print("Final balance = {}".format(balance.value))

if __name__ == "__main__":
    for _ in range(10):
        perform_transactions()
```

以下是一种错误写法：

```
import multiprocessing

def withdraw(balance):
    for _ in range(10000):
        with multiprocessing.Lock():
            balance.value = balance.value - 1


def deposit(balance):
    for _ in range(10000):
        with multiprocessing.Lock():
            balance.value = balance.value + 1
```

```
def perform_transactions():
    balance = multiprocessing.Value('i', 100)

    p1 = multiprocessing.Process(target=withdraw, args=(balance,))
    p2 = multiprocessing.Process(target=deposit, args=(balance,))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
    print("Final balance = {}".format(balance.value))

if __name__ == "__main__":
    for _ in range(10):
        perform_transactions()
"""
out:
    Final balance = 84
    Final balance = 84
    Final balance = 114
    Final balance = 143
    Final balance = 91
    Final balance = 79
    Final balance = 173
    Final balance = 94
    Final balance = 99
    Final balance = 91
"""
```

4.5 进程池化

```
import multiprocessing
import os

def square(n):
    print("Worker process id for {0}: {1}".format(n, os.getpid()))
    return (n*n)

if __name__ == "__main__":
    mylist = [1,2,3,4,5]

    p = multiprocessing.Pool()

    result = p.map(square, mylist)
    print(result)

"""
out:
    Worker process id for 1: 8300
    Worker process id for 2: 13072
    Worker process id for 3: 8300
    Worker process id for 4: 8300
    Worker process id for 5: 8300
    [1, 4, 9, 16, 25]
"""
```

我们还可以向Pool传递其他参数，比如：

- processes：用于指定分配的进程数量
- maxtasksperchild：用于指定每个子进程所分配到的最大任务数

我们还可以用这些参数对池中的进程进行一些初始化：

- initializer：为进程指定初始化函数
- initargs：向初始化函数传递的参数值

在创建了Pool对象之后，我们只需要使用`map()`方法，使其执行具体的任务即可。

4.6 多进程运行含有任意个参数的函数

这个博客讲的很**NICE:**[Python：多进程运行含有任意个参数的函数_Jayce~的博客-CSDN博客]